



Machine Learning for Finance
Project Report

NEURAL NETWORK BASED EUROPEAN OPTION PRICING

UNDER BLACK-SCHOLES SDE DYNAMICS

*A supervised learning study of neural networks
as pricing surrogate models*

MATTEO GIORGI
LORENZO GHIOTTI
NALUMANSI ANNE FRANCES
ALICE PEDRON

May 30, 2026

Abstract

This project studies whether a feed-forward neural network can accurately approximate the Black-Scholes pricing function for European call options. The problem is addressed in a controlled setting: data are generated synthetically from the analytical Black-Scholes formula, while Monte Carlo simulation is used as a numerical benchmark. The neural network is trained in a supervised learning framework to learn the mapping from market and contract parameters to the theoretical option price. The goal is not to build a real trading system, but to evaluate the role of a neural network as a pricing surrogate in a mathematically well-defined financial setting.

Contents

Abstract	ii
1 Introduction	1
1.1 Motivation	1
1.2 Project Goal	1
1.3 Research Question	1
1.4 Contributions	2
1.5 Report Structure	2
2 Financial and Mathematical Background	3
2.1 European Call Options	3
2.2 Stochastic Differential Equations	3
2.3 Geometric Brownian Motion	3
2.4 Black-Scholes Model	4
2.5 Black-Scholes Formula	4
2.6 Monte Carlo Pricing	4
3 Machine Learning Background	5
3.1 Supervised Learning for Regression	5
3.2 Neural Networks as Function Approximators	5
3.3 Feed-Forward Neural Network Architecture	5
3.4 Support Vector Regression	6
3.5 Pricing Surrogate Models	6
4 Dataset and Experimental Design	7
4.1 Synthetic Dataset Generation	7
4.2 Input Variables and Target	7
4.3 Feature Engineering: Moneyness	7
4.4 Train/Validation/Test Split	8
4.5 Clean and Noisy Target Settings	8
4.6 Reproducibility	8
5 Methodology	9
5.1 Baseline Black-Scholes Pricing	9
5.2 Monte Carlo Benchmark	9

5.3	Neural Network Model	9
5.4	Training Procedure	9
5.5	Evaluation Metrics	10
5.6	Runtime Benchmark	10
5.7	SVR Baseline	10
5.8	Robustness Experiments with Noisy Targets	10
6	Software Implementation	12
6.1	Repository Structure	12
6.2	Configuration System	12
6.3	Experimental Pipeline	13
6.4	Testing and Documentation	13
6.5	Computational Efficiency	14
7	Experimental Results	15
7.1	Final Neural Network Performance	15
7.2	Monte Carlo Comparison	16
7.3	Feature Engineering Results	16
7.4	Activation Function Results	17
7.5	Runtime Results	17
7.6	SVR Benchmark Results	17
7.7	Noisy Target Robustness Results	18
7.8	Error Analysis	19
8	Discussion	20
8.1	Interpretation of Results	20
8.2	When the Neural Network Works Well	20
8.3	Sources of Error	20
8.4	Limits of the Synthetic Setting	20
8.5	Comparison with Classical Methods	21
9	Conclusions and Future Work	22
9.1	Conclusions	22
9.2	Limitations	22
9.3	Future Extensions	22

INTRODUCTION

1.1 Motivation

Derivative pricing is one of the classical problems in quantitative finance. For European options, the Black-Scholes model provides a closed-form price under precise assumptions on the dynamics of the underlying asset, market completeness, and constant volatility [1]. The availability of an analytical solution makes Black-Scholes an ideal controlled environment for evaluating numerical methods and machine learning models.

At the same time, many relevant pricing problems do not admit simple closed-form solutions. In such cases, classical numerical methods such as Monte Carlo simulation can become computationally expensive, especially when prices must be evaluated repeatedly across many contracts or parameter configurations. This motivates the study of neural networks as pricing surrogate models: after training, a neural network can approximate a pricing map through fast inference.

1.2 Project Goal

The goal of this project is to study whether a feed-forward neural network can approximate the Black-Scholes pricing function for European call options. The project is not intended to build a trading system or to predict real market prices. Instead, it focuses on a mathematically controlled supervised learning problem where the target function is known exactly.

1.3 Research Question

The main research question is:

Can a feed-forward neural network accurately approximate the Black-Scholes pricing function for European call options?

The project compares three main pricing approaches:

- the analytical Black-Scholes formula, used as the ground truth;
- Monte Carlo simulation, used as a classical numerical benchmark;
- a feed-forward neural network, used as a supervised learning model and pricing surrogate.

Additional experiments consider feature engineering, activation functions, runtime performance, a Support Vector Regression baseline, and robustness to controlled label noise.

1.4 Contributions

The main contributions of the project are:

- construction of a reproducible synthetic Black-Scholes dataset;
- implementation of an analytical Black-Scholes benchmark and a Monte Carlo benchmark;
- training and evaluation of a feed-forward neural network pricing surrogate;
- comparison of model variants based on moneyness and activation functions;
- evaluation of runtime differences between analytical pricing, neural inference, and Monte Carlo simulation;
- inclusion of a reduced-scale SVR baseline;
- robustness experiments with controlled noisy Black-Scholes targets;
- modular Python implementation with tests, documentation, and reproducible experiment artifacts.

1.5 Report Structure

The report is organized as follows. Chapter 2 introduces the financial and mathematical background. Chapter 3 summarizes the machine learning concepts used in the project. Chapter 4 describes the dataset and experimental design. Chapter 5 presents the methodology and benchmark models. Chapter 6 describes the software implementation. Chapter 7 reports the experimental results. Chapter 8 discusses the interpretation and limitations of the results. Chapter 9 concludes and outlines possible future extensions.

FINANCIAL AND MATHEMATICAL BACKGROUND

2.1 European Call Options

A European call option gives its holder the right, but not the obligation, to buy an underlying asset at a fixed strike price K at maturity T . If the terminal asset price is S_T , the payoff is:

$$\max(S_T - K, 0). \quad (2.1)$$

The option value depends on the current asset price, the strike price, time to maturity, the risk-free rate, and the volatility of the underlying asset. General arbitrage-pricing principles for derivative securities can be introduced in discrete-time models before moving to continuous-time diffusion models [5, 4]. In this project, only European call options are considered.

2.2 Stochastic Differential Equations

Continuous-time asset price models are often described by stochastic differential equations. An SDE combines a deterministic drift component with a random diffusion component driven by Brownian motion. This framework is standard in financial modeling because asset prices evolve continuously but are subject to uncertainty [6].

2.3 Geometric Brownian Motion

In the Black-Scholes model, the underlying asset follows a geometric Brownian motion under the risk-neutral measure:

$$dS_t = rS_t dt + \sigma S_t dW_t, \quad (2.2)$$

where S_t is the asset price at time t , r is the risk-free interest rate, σ is the constant volatility, and W_t is a standard Brownian motion.

The solution at maturity T is:

$$S_T = S_0 \exp \left[\left(r - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z \right], \quad Z \sim \mathcal{N}(0, 1). \quad (2.3)$$

2.4 Black-Scholes Model

The Black-Scholes model assumes frictionless markets, constant volatility, a constant risk-free rate, no arbitrage, and continuous trading. Under these assumptions, the price of a European option can be derived from risk-neutral valuation.

For a European call, the value is the discounted expected payoff:

$$C = e^{-rT} \mathbb{E}^{\mathbb{Q}} [\max(S_T - K, 0)]. \quad (2.4)$$

2.5 Black-Scholes Formula

The analytical Black-Scholes price of a European call option is:

$$C_{BS} = S_0 N(d_1) - K e^{-rT} N(d_2), \quad (2.5)$$

where $N(\cdot)$ is the cumulative distribution function of a standard normal random variable and:

$$d_1 = \frac{\ln(S_0/K) + (r + \frac{1}{2}\sigma^2) T}{\sigma\sqrt{T}}, \quad (2.6)$$

$$d_2 = d_1 - \sigma\sqrt{T}. \quad (2.7)$$

In this project, C_{BS} is used both as the supervised learning target and as the reference value for evaluation. This is what makes the experiment controlled: the true target function is known analytically.

2.6 Monte Carlo Pricing

Monte Carlo simulation approximates the risk-neutral expectation by generating many realizations of S_T :

$$\hat{C}_{MC} = e^{-rT} \frac{1}{M} \sum_{i=1}^M \max(S_T^{(i)} - K, 0). \quad (2.8)$$

For sufficiently large M , $\hat{C}_{MC}$ converges to C_{BS} . In this project, Monte Carlo is used as a numerical benchmark and as a comparison point for the neural network. Since the terminal distribution of S_T is known exactly under Black-Scholes, the implementation simulates terminal prices directly rather than discretizing the full path.

3

MACHINE LEARNING BACKGROUND

3.1 Supervised Learning for Regression

The pricing task is formulated as a supervised regression problem. Each observation consists of option and market parameters, and the target is the corresponding Black-Scholes call price. Given a dataset $\{(x_i, y_i)\}_{i=1}^n$, the goal is to learn a function f_θ such that:

$$f_\theta(x_i) \approx y_i. \quad (3.1)$$

In this project, the target y_i is not observed from markets. It is generated by the analytical Black-Scholes formula, which allows direct measurement of approximation error.

3.2 Neural Networks as Function Approximators

Feed-forward neural networks are flexible nonlinear function approximators. Under general assumptions, sufficiently large feed-forward networks can approximate continuous functions on compact domains [2, 3]. This makes them natural candidates for approximating a deterministic pricing function such as Black-Scholes.

The purpose of the neural network is not to replace the analytical formula when that formula is available. Rather, the Black-Scholes setting provides a clean benchmark for studying the surrogate-model idea before moving to models where closed-form prices may not exist.

3.3 Feed-Forward Neural Network Architecture

The model used in the project is a Multi-Layer Perceptron. It receives numerical input features and returns one scalar output, the predicted option price.

The baseline architecture is:

$$5 \rightarrow 64 \rightarrow 64 \rightarrow 32 \rightarrow 1, \quad (3.2)$$

where the five inputs are (S_0, K, T, r, σ) . After intermediate experiments, the selected final configuration uses six input features, including moneyness, and SiLU hidden activations:

$$6 \rightarrow 64 \rightarrow 64 \rightarrow 32 \rightarrow 1. \quad (3.3)$$

The final neural network approximates:

$$f_{\theta}(S_0, K, T, r, \sigma, S_0/K) \approx C_{BS}. \quad (3.4)$$

3.4 Support Vector Regression

Support Vector Regression is included as an additional classical machine learning baseline. The project uses SVR with an RBF kernel, which is able to model nonlinear relationships in tabular data.

SVR is evaluated on reduced datasets because kernel methods generally scale less favorably than neural networks as the number of training observations increases. For this reason, the SVR experiments should be interpreted as a classical reference point rather than as a full-scale replacement for the neural network pipeline.

3.5 Pricing Surrogate Models

A pricing surrogate is a model trained to approximate a pricing function. Once trained, the surrogate can produce prices quickly for new inputs. This is useful when the original pricing method is computationally expensive, for example when repeated Monte Carlo simulations are required.

In the present Black-Scholes setting, the analytical formula remains the fastest and most exact method. The relevant surrogate comparison is therefore mainly between neural network inference and Monte Carlo simulation.

DATASET AND EXPERIMENTAL DESIGN

4.1 Synthetic Dataset Generation

The dataset is generated by randomly sampling option and market parameters from fixed ranges. Each observation initially contains the primitive Black-Scholes input variables:

$$x = (S_0, K, T, r, \sigma), \quad (4.1)$$

and the target is:

$$y = C_{BS}(S_0, K, T, r, \sigma). \quad (4.2)$$

Using synthetic data is a methodological choice. The project studies whether a neural network can approximate a known mathematical pricing function, not whether it can fit noisy market option prices. With synthetic Black-Scholes data, the ground truth is exact and reproducible.

4.2 Input Variables and Target

The parameter ranges used to generate the dataset are reported in Table 4.1.

Variable	Description	Range
S_0	initial underlying price	[50, 150]
K	strike price	[50, 150]
T	maturity in years	[0.1, 2]
r	risk-free rate	[0, 0.05]
σ	volatility	[0.1, 0.6]

Table 4.1: Parameter ranges used to generate the synthetic dataset.

The target variable is the European call option price computed using the analytical Black-Scholes formula. Therefore, each supervised learning observation has the form:

$$(S_0, K, T, r, \sigma) \mapsto C_{BS}. \quad (4.3)$$

4.3 Feature Engineering: Moneyness

The final model augments the input vector with moneyness:

$$\text{moneyness} = \frac{S_0}{K}. \quad (4.4)$$

The final input vector is:

$$x_{\text{final}} = (S_0, K, T, r, \sigma, S_0/K). \quad (4.5)$$

Moneyness does not add new information beyond S_0 and K , but it exposes a financially meaningful relationship directly to the model. Intermediate experiments showed that this engineered feature can improve absolute-error metrics.

4.4 Train/Validation/Test Split

The dataset is split into training, validation, and test sets. The training set is used to optimize the model parameters. The validation set is used for early stopping and model selection during training. The test set is kept separate and is used only for final evaluation.

All splits are controlled by fixed random seeds. This makes the reported results reproducible and ensures that different model variants can be compared under consistent conditions.

4.5 Clean and Noisy Target Settings

The main experiment uses clean Black-Scholes targets. The project also includes a controlled noisy-target setting used for robustness analysis. In that setting, the input variables are unchanged, while the training labels are perturbed.

For each clean price C_{BS} , the noisy target is generated as:

$$\tilde{C} = \max(C_{\text{BS}} + \varepsilon, 0), \quad \varepsilon \sim \mathcal{N}\left(0, (\ell \max(C_{\text{BS}}, 1))^2\right), \quad (4.6)$$

where ℓ is the noise level. The evaluation target remains the clean analytical Black-Scholes price. This experiment measures robustness to imperfect labels while preserving a known ground truth.

4.6 Reproducibility

Reproducibility is handled through fixed random seeds and explicit configuration objects. Command-line options define each run, and the corresponding artifacts are saved to disk. Generated datasets and model checkpoints are stored in reproducible output directories. Selected final metrics and figures are tracked in the repository.

METHODOLOGY

5.1 Baseline Black-Scholes Pricing

The analytical Black-Scholes formula is the reference pricing method. It is used to generate supervised learning labels and to evaluate prediction errors. Because the formula is available in closed form, it provides an exact ground truth for the controlled experiment.

5.2 Monte Carlo Benchmark

Monte Carlo simulation is used as a numerical benchmark. For each option, terminal prices are sampled from the exact Black-Scholes terminal distribution and discounted payoffs are averaged. The benchmark is evaluated against the same analytical Black-Scholes reference as the neural network.

The final Monte Carlo benchmark uses 50,000 simulated paths on a deterministic subset of 512 test options. The subset restriction keeps the benchmark computationally bounded while still providing a meaningful comparison with a classical simulation-based pricing method.

5.3 Neural Network Model

The neural network is a feed-forward MLP for regression. The selected final configuration uses:

- input features $(S_0, K, T, r, \sigma, S_0/K)$;
- hidden layers of widths 64, 64, and 32;
- SiLU activation functions in the hidden layers;
- one scalar output representing the predicted call price.

The optimized loss is the mean squared error:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x_i) - y_i)^2. \quad (5.1)$$

5.4 Training Procedure

Inputs are standardized using **StandardScaler**. The target is also standardized during training to improve numerical stability and convergence. Predictions are transformed back to original price units before evaluation.

The network is trained with Adam, mini-batches, weight decay, and early stopping based on validation loss. The final experiment uses up to 200 epochs and a batch size of 1024. After prediction, prices are clipped at zero to avoid financially invalid negative option prices.

5.5 Evaluation Metrics

The main metrics are:

- Mean Absolute Error (MAE);
- Root Mean Squared Error (RMSE);
- coefficient of determination R^2 ;
- MAPE computed only on prices greater than 1.

The MAPE restriction avoids unstable percentage errors for options whose theoretical price is close to zero.

5.6 Runtime Benchmark

The runtime benchmark separates training time from pricing time. This distinction is important because a neural network has an upfront training cost, while its surrogate value is measured during repeated inference.

The benchmark compares:

- vectorized analytical Black-Scholes pricing;
- neural network inference after training;
- Monte Carlo pricing.

5.7 SVR Baseline

The project includes a reduced-scale Support Vector Regression baseline with an RBF kernel. The SVR benchmark uses 5,000 synthetic samples per run and is repeated over three random seeds. The same engineered feature set used by the final neural network, including moneyness, is used for SVR.

Both input features and target prices are standardized before fitting. Predictions are transformed back to original price units before computing metrics. SVR is kept on reduced datasets because kernel methods scale less favorably than neural networks.

5.8 Robustness Experiments with Noisy Targets

The robustness experiments perturb only the training labels, not the input variables. Models are trained on noisy Black-Scholes prices and evaluated against clean analytical Black-Scholes prices.

The neural network noisy-target experiment uses 50,000 synthetic contracts per noise level. The same noisy-target mechanism is also applied to the reduced-scale SVR baseline, using 5,000 contracts per run and three random seeds.

These experiments are controlled label-noise tests. They should not be interpreted as real market-price modeling, because real exchange-traded options include additional effects such as bid-ask spreads, liquidity, dividends, and implied-volatility structure.

SOFTWARE IMPLEMENTATION

6.1 Repository Structure

The project is implemented in Python with a modular structure. The main libraries are NumPy, pandas, SciPy, scikit-learn, matplotlib, and PyTorch.

```
src/nn_option_pricing/
  black_scholes.py      # analytical formula and payoff
  config.py             # experiment configuration dataclasses
  dataset.py           # synthetic data generation and storage
  evaluation.py         # regression metrics
  model.py             # PyTorch feed-forward neural network
  monte_carlo.py        # efficient Monte Carlo simulation
  noise.py             # controlled target-noise generation
  noisy_experiment.py   # neural-network noisy-target experiment
  noisy_svr_experiment.py # noisy-target SVR benchmark
  pipeline.py          # end-to-end experiment
  plots.py             # visualizations
  svr.py               # reduced-scale SVR benchmark
  training.py          # training loop, scaling, early stopping
scripts/
  run_experiment.py     # main experiment CLI
  benchmark_runtime.py  # runtime benchmark CLI
  run_svr_benchmark.py  # SVR benchmark CLI
  run_noisy_targets_experiment.py
  run_noisy_svr_benchmark.py
```

Listing 6.1: Main project structure.

The reusable scientific and machine learning logic is contained in the package **src/nn_option_pricing/**. The files in **scripts/** are command-line entry points that build configurations and call functions from the package.

6.2 Configuration System

The project uses dataclasses to keep experiment parameters explicit. The configuration includes dataset ranges, training parameters, Monte Carlo parameters, and output paths. This avoids hard-coded constants scattered across the codebase and makes experiments easier to reproduce. Each experiment saves a configuration snapshot in JSON format. This makes it possible to recover the exact settings used to produce a given result.

6.3 Experimental Pipeline

The main pipeline performs the following steps:

1. create output directories;
2. generate the synthetic dataset;
3. save the dataset and configuration;
4. split the data into training, validation, and test sets;
5. fit input and target scalers;
6. train the neural network with early stopping;
7. predict on the held-out test set;
8. compute neural-network metrics;
9. run the Monte Carlo benchmark;
10. save metrics and diagnostic figures.

The final experiment is run through:

```
.venv/bin/python scripts/run_experiment.py \  
  --n-samples 100000 \  
  --max-epochs 200 \  
  --batch-size 1024 \  
  --mc-n-paths 50000 \  
  --mc-evaluation-samples 512 \  
  --feature-set with_moneyness \  
  --activation silu \  
  --seed 42 \  
  --data-dir data/final_improved \  
  --output-dir outputs/final_improved
```

Listing 6.2: Command used for the final experiment.

6.4 Testing and Documentation

The test suite covers:

- analytical Black-Scholes pricing;
- synthetic dataset generation;
- model construction;
- Monte Carlo pricing;
- target-noise generation;
- SVR benchmarks;
- end-to-end pipeline smoke tests.

The project also includes Sphinx documentation generated from narrative documentation and Python docstrings. This supports code readability and makes the implementation easier to inspect.

6.5 Computational Efficiency

The Black-Scholes formula and dataset generation are vectorized with NumPy. The Monte Carlo method uses the exact terminal distribution of the geometric Brownian motion, avoiding unnecessary time discretization. Monte Carlo simulation is batched both over options and over simulated paths, which keeps memory usage under control.

The neural network training uses PyTorch **DataLoader**, mini-batch gradient descent, input standardization, and target scaling. Prediction is performed in batches under **torch.no_grad()**. These choices make the code suitable both for quick development runs and for larger experiments.

EXPERIMENTAL RESULTS

7.1 Final Neural Network Performance

The final neural network results on the held-out test set are reported in Table 7.1.

Metric	Value
MAE	0.04290
RMSE	0.06208
R^2	0.999993
MAPE, prices > 1	0.4803%

Table 7.1: Final neural network metrics against analytical Black-Scholes prices.

The results show that the feed-forward neural network with moneyness as an engineered feature and SiLU hidden activations approximates the Black-Scholes pricing function with very high accuracy on the sampled parameter domain. The coefficient of determination is close to one, while MAE and RMSE are small in absolute price units.



Figure 7.1: Training loss and validation loss for the final experiment.

Figure 7.1 shows that the optimization process remains stable. The validation curve follows the training curve closely, suggesting that the model does not simply memorize the training set.

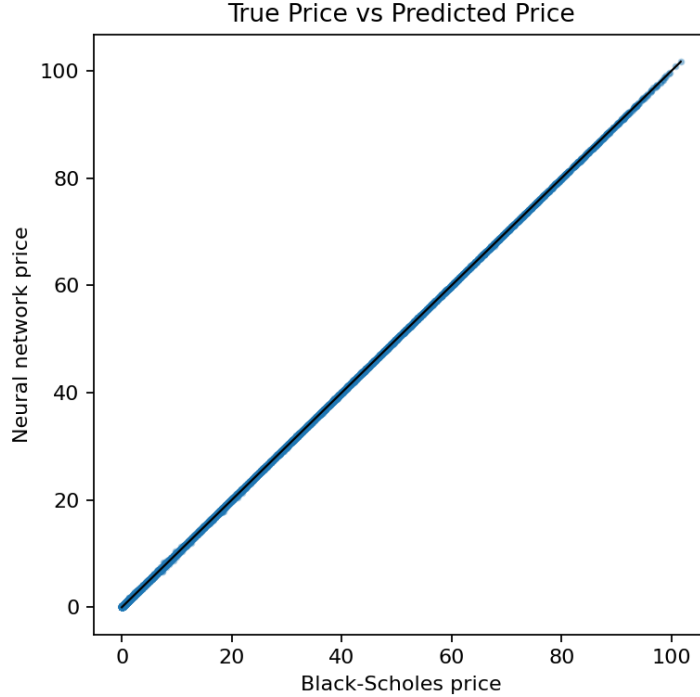


Figure 7.2: Analytical Black-Scholes prices against neural network predictions.

Figure 7.2 confirms that predicted prices are tightly aligned with analytical prices across the observed price range.

7.2 Monte Carlo Comparison

The Monte Carlo benchmark is evaluated against the analytical Black-Scholes price on 512 deterministic test options. The results are shown in Table 7.2.

Metric	Value
MAE	0.08882
RMSE	0.15790
R^2	0.999951
MAPE, prices > 1	0.6483%

Table 7.2: Final Monte Carlo metrics against analytical Black-Scholes prices.

The Monte Carlo benchmark also achieves high accuracy, as expected when the number of simulated paths is large. However, it remains a simulation-based estimator and therefore has sampling error. The neural network, in contrast, produces deterministic forward-pass predictions once trained.

7.3 Feature Engineering Results

An intermediate experiment compared the baseline feature set with the engineered moneyiness feature. Adding moneyiness reduced MAE from 0.17269 to 0.15417 in the controlled interme-

diate setting with ReLU activations. This suggests that exposing the ratio S_0/K directly can help optimization, even though it does not add information beyond S_0 and K .

7.4 Activation Function Results

Another intermediate experiment compared ReLU, Tanh, and SiLU hidden activations using the same baseline feature set. SiLU achieved the lowest MAE and RMSE among the tested activation functions. This result is consistent with the smoothness of the Black-Scholes pricing function and motivated the final choice of SiLU.

7.5 Runtime Results

The runtime benchmark separates the initial neural network training cost from pricing time after training. Using the improved final configuration, training required approximately 359 seconds in the benchmark environment. Once trained, the neural network priced 1,000 options in approximately 0.0319 seconds on average. The Monte Carlo benchmark with 20,000 paths required approximately 1.2824 seconds for the same number of options, while the vectorized analytical Black-Scholes formula required approximately 0.00091 seconds.

Method	Mean time for 1,000 options	Mean options/s
Black-Scholes formula	0.00091 s	1,101,094.82
Neural network inference	0.03190 s	31,350.77
Monte Carlo, 20,000 paths	1.28235 s	779.82

Table 7.3: Runtime benchmark for analytical, neural-network, and Monte Carlo pricing.

As expected, the analytical Black-Scholes formula is the fastest method when available. The relevant surrogate-pricing comparison is therefore between neural network inference and Monte Carlo simulation. In this benchmark, neural inference is roughly 40 times faster than Monte Carlo after the model has been trained.

7.6 SVR Benchmark Results

The reduced-scale SVR benchmark provides an additional comparison with a classical supervised regression model. The experiment was repeated over three random seeds, using 5,000 synthetic contracts per run and the same engineered moneyness feature used by the final neural network. Aggregate results are reported in Table 7.4.

Metric	Mean	Standard deviation
MAE	0.15088	0.00732
RMSE	0.23100	0.00592
R^2	0.999905	0.000005
MAPE, prices > 1	1.8253%	0.2422%
Fit time	12.3198 s	0.7977 s
Prediction time	0.0301 s	0.0029 s

Table 7.4: Reduced-scale SVR benchmark aggregated over three random seeds.

The SVR baseline also approximates the Black-Scholes pricing function well on reduced datasets. This is indicated by its high R^2 . However, its absolute error is higher than that of the final neural network configuration.

7.7 Noisy Target Robustness Results

Table 7.5 reports the robustness experiment in which the neural network is trained on noisy Black-Scholes prices and evaluated against the clean analytical Black-Scholes target.

Noise level	MAE	RMSE	R^2	MAPE, prices > 1
0%	0.10254	0.15169	0.999958	1.3213%
1%	0.11084	0.16105	0.999953	1.3960%
5%	0.17827	0.24167	0.999893	2.0943%

Table 7.5: Noisy-target neural-network results evaluated against clean Black-Scholes prices.

The results show the expected degradation as label noise increases. Nevertheless, the model remains accurate even under moderate target noise.

The same noisy-target experiment was also repeated with the reduced-scale SVR baseline. Table 7.6 reports aggregate metrics against clean Black-Scholes prices.

Noise level	MAE	RMSE	R^2	MAPE, prices > 1
0%	0.15088	0.23100	0.999905	1.8253%
1%	0.19284	0.30765	0.999830	1.8705%
5%	0.56185	1.05057	0.998025	3.4903%

Table 7.6: Noisy-target SVR benchmark evaluated against clean Black-Scholes prices.

SVR remains accurate under mild label noise, but the degradation is stronger at the 5% noise level. Since the SVR benchmark is run on reduced datasets, this comparison is best interpreted as an additional classical robustness reference rather than as a direct replacement for the neural network experiment.

7.8 Error Analysis

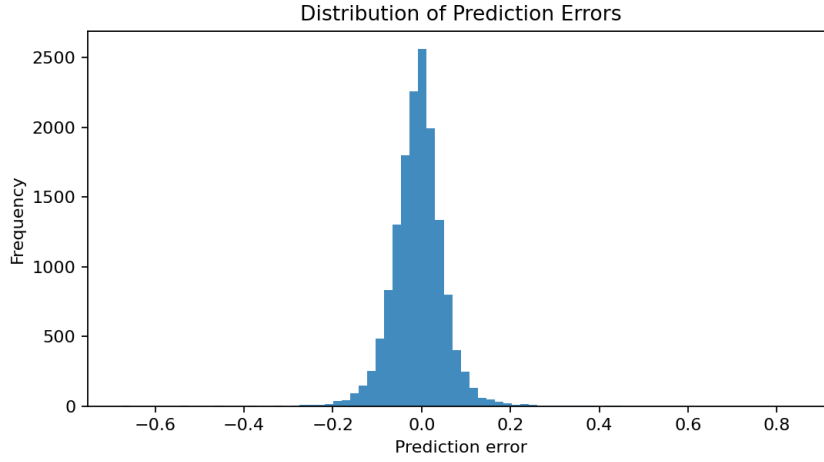


Figure 7.3: Distribution of neural network prediction errors.

The error distribution in Figure 7.3 is concentrated near zero, which is consistent with the low MAE and RMSE values reported in Table 7.1.

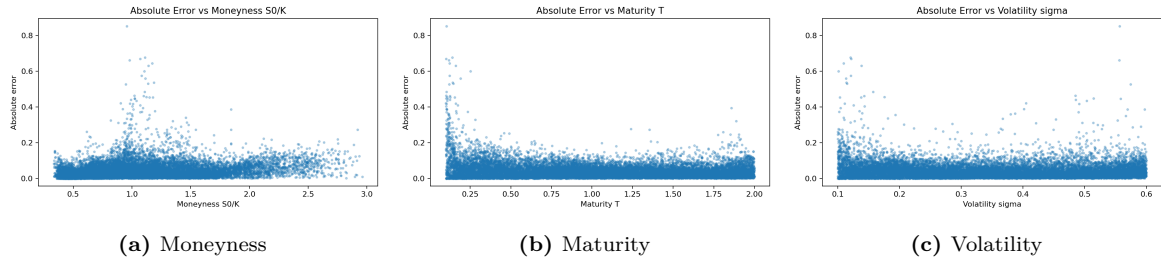


Figure 7.4: Absolute neural network error against selected financial features.

Figure 7.4 is used to inspect whether the approximation error changes systematically across the parameter space. This is important because aggregate metrics alone may hide localized weaknesses, especially around different moneyness regimes, short maturities, or high-volatility regions.

DISCUSSION

8.1 Interpretation of Results

The final results indicate that the neural network can approximate the Black-Scholes pricing function with high accuracy in the sampled parameter domain. This supports the main research question in the controlled synthetic setting considered in the project.

The comparison with Monte Carlo should be interpreted carefully. The neural network is evaluated on the full held-out test set, whereas Monte Carlo is evaluated on a deterministic subset due to computational cost. The purpose of the comparison is therefore not to declare a universally superior method, but to clarify the behavior of a learned surrogate relative to a classical simulation-based estimator.

8.2 When the Neural Network Works Well

The neural network works well when test samples come from the same parameter ranges used during training. In this regime, the mapping from option parameters to Black-Scholes prices is smooth and deterministic. The engineered moneyness feature and the SiLU activation function further improve the approximation in the experiments.

The runtime benchmark also highlights a practical advantage of the surrogate approach. After training, neural inference is substantially faster than Monte Carlo simulation for repeated pricing queries in the benchmark setting.

8.3 Sources of Error

Prediction errors can arise from finite training data, imperfect optimization, model capacity limits, and numerical scaling choices. Errors may also vary across the parameter space. For this reason, the project includes diagnostic plots against moneyness, maturity, and volatility.

The noisy-target experiments show another source of error: imperfect labels. When training targets are perturbed, the model remains accurate under moderate noise, but approximation quality decreases as the noise level increases. This behavior is expected because the model receives less precise supervision.

8.4 Limits of the Synthetic Setting

The use of synthetic data is appropriate for the research question, because the project aims to approximate the Black-Scholes pricing function itself. However, this also limits the scope of

the conclusions.

Real exchange-traded option prices reflect additional effects such as bid-ask spreads, liquidity, dividends, supply and demand, and implied-volatility structure. Training on such data would change the nature of the problem from approximating Black-Scholes to modeling market prices. Therefore, the results should not be interpreted as evidence that the model can price real market options.

8.5 Comparison with Classical Methods

The analytical Black-Scholes formula remains the exact and fastest method in this specific setting. Monte Carlo is useful as a numerical benchmark and as a method that generalizes to settings without closed-form solutions. SVR provides a classical machine learning baseline on reduced datasets.

The neural network occupies a different role. It is not intended to outperform the closed-form formula when the formula is available. Its value lies in demonstrating how a supervised model can learn a known pricing map and act as a fast surrogate after training.

CONCLUSIONS AND FUTURE WORK

9.1 Conclusions

The final experiment confirms that a feed-forward neural network can approximate the Black-Scholes pricing function for European call options with high accuracy in a controlled synthetic environment. With 100,000 synthetic contracts, the network achieves an MAE of 0.04290, an RMSE of 0.06208, and an R^2 of 0.999993 on the held-out test set. These results answer the research question positively within the assumptions and parameter ranges considered in the project.

The comparison with Monte Carlo clarifies the role of the neural network. Monte Carlo is a classical numerical estimator and remains useful as a benchmark, while the trained neural network acts as a deterministic pricing surrogate. The runtime benchmark further supports the surrogate interpretation: after training, neural inference is substantially faster than Monte Carlo simulation for repeated pricing queries.

The additional SVR experiment provides a classical machine learning reference point on reduced datasets. The noisy-target experiments show that both the neural network and the SVR baseline remain accurate under controlled label perturbations, although approximation quality decreases as the noise level increases.

9.2 Limitations

The main limitations are:

- the data are synthetic and generated under Black-Scholes assumptions;
- only European call options are considered;
- the target pricing function is known analytically;
- noisy targets are controlled perturbations and should not be interpreted as real market prices;
- generalization outside the sampled parameter ranges is not guaranteed.

These limitations are not flaws in the experiment, but boundaries of the research question. The project deliberately studies a controlled approximation problem before moving to more complex financial settings.

9.3 Future Extensions

Natural future extensions include:

- compare additional neural network architectures beyond the selected MLP;
- tune and evaluate further classical machine learning baselines on controlled reduced datasets;
- study the error in specific regions of the parameter space, for instance deep in-the-money or out-of-the-money options;
- extend the framework to stochastic volatility models such as Heston;
- price more complex derivatives such as basket options or American options;
- estimate Greeks using either automatic differentiation or finite differences;
- investigate real or pseudo-real option data while explicitly changing the research question to market-price modeling.

These extensions would generalize the approach beyond the Black-Scholes benchmark while preserving the experimental discipline developed in the current project.

Bibliography

- [1] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- [2] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989.
- [3] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- [4] Andrea Pascucci and Wolfgang J. Runggaldier. *Financial Mathematics: Theory and Problems for Multi-period Models*. UNITEXT. Springer Milan, 2012.
- [5] Steven E. Shreve. *Stochastic Calculus for Finance I: The Binomial Asset Pricing Model*. Springer Finance. Springer New York, 2004.
- [6] Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer Finance. Springer New York, 2004.